



[팀장님 질문] WHY SSE는 ping을 날리는데 HTTP API 요청하는 것과 뭐가 다른 것인가?

[SSE vs HTTP Polling 정리](#)

[HTTP API 요청의 동작 과정](#)

[그럼 SSE는 어떻게 동작하지?](#)

[SSE와 WebSocket 비교](#)

[SSE 안정성 검증 포인트](#)

SSE vs HTTP Polling 정리

기본적인 개념부터 비교해보자

HTTP API(Polling)

- 요청/응답 1회 후 연결 종료
- 새로운 요청마다 TCP + TLS 핸드셰이크 재실행
- HTTP 헤더 오버헤드 매번 발생

SSE(Server-Sent Events)

- 최초 한 번 HTTP 연결 후 유지
- 서버 → 클라이언트 단방향 이벤트 스트리밍 가능
- 오버헤드 최소화, 지연 시간 단축

그럼 HTTP API와 SSE가 다른 부분이 "TCP + TLS 핸드셰이크"의 여러번 반복인데 이 과정인지 자세하게 알아보자!

→ 사실 아마 CS 면접이든 네트워크 시간에 자세하게 배워봤을테니 너무나도 잘 알고 계시겠지만 여기서는 간략하게 과정만 정리하고 넘어가겠습니다.

HTTP API 요청의 동작 과정

1. TCP 연결 (3-way Handshake)

- SYN → SYN-ACK → ACK
2. TLS Handshake (HTTPS)
 - 인증서 교환, 암호화 방식 협상, 세션 키 설정
 3. HTTP 요청/응답
 - 요청 헤더 전송
 - 서버 처리 → 응답(헤더+바디) 전송
 4. 연결 종료
 - 기본적으로 서버에서 FIN으로 세션 종료
 - keep-alive 시 잠시 유지 가능, idle timeout 이후 종료

→ Polling의 경우는 위 과정을 매번 반복 → 네트워크·서버 리소스 소모 증가, 지연 발생

그럼 SSE는 어떻게 동작하지?

1. 클라이언트 → 서버: GET 요청 + Accept: text/event-stream 헤더
2. 서버 → 클라이언트: Content-Type: text/event-stream 응답
3. 서버는 HTTP 응답 바디를 열어둔 상태로 유지
4. 새로운 이벤트 발생 시 data: 라인 추가 전송
5. 서버가 FIN 보내기 전까지 TCP 연결 유지

→ 최초 연결 이후 추가 핸드셰이크나 헤더 오버헤드 없음

그래서 예전에 POLLING 방식을 제안드린 이유!!!

musicgen만 있었던 경우에 단 한 번 혹은 많아도 두번의 요청으로 응답이 올 것이라고 판단

GPU 기준 10초면 충분히 배경음을 생성하니 굳이 sse 연결을 해야 할까라고 판단을 해 polling으로 제안을 드렸던 것

SSE도 결국 연결을 위해 HTTP API와 같은 작업은 무조건 1회 진행을 하는데

한 번의 요청으로 응답이 온다면 굳이 SSE가 아닌 Polling 방식이 적합하지 않을까 판단

BUT 결론적으로 다른 경우들도 있을 가능성이 있고 여러 리스크를 고려해 sse를 도입하기로 결정 - 🍏

SSE(Server-Sent Events) 선택 배경 및 의사결정 과정

정리

결국 다른 점은 연결을 하기 위한 작업들이 생략이 된다. SSE를 할 경우, musicgen만 있을 때는 모르겠지 만 지금 감정 추출이 들어온 것까지 생각하면 polling보다는 SSE가 적합하다고 판단

자 그렇다면 SSE 특징인 단방향에 초점을 맞춰보고 싶은데 특히 웹소켓과 SSE 차이를 비교해보자

SSE와 WebSocket 비교

- **SSE**
 - 단방향 (서버 → 클라이언트)
 - HTTP 기반, 텍스트 스트리밍 전용
 - 연결 유지 비용이 적음
- **WebSocket**
 - 양방향 (클라이언트 ↔ 서버)
 - Upgrade 헤더로 HTTP → WS 프로토콜 변경
 - 이진 데이터도 전송 가능

다시 SSE 동작 방식을 떠올려보자

1. 클라이언트 → 서버: GET 요청 + Accept: text/event-stream 헤더
2. 서버 → 클라이언트: Content-Type: text/event-stream 응답
3. 서버는 HTTP 응답 바디를 열어둔 상태로 유지
4. 새로운 이벤트 발생 시 data: 라인 추가 전송
5. 서버가 FIN 보내기 전까지 TCP 연결 유지

연결을 끝내는 주체가 서버에 있고, 서버에서 클라이언트에게 요청을 보내는 것이니 기존 HTTP 프로토콜 위에서도 충분히 동작을 할 수 있습니다.

반면에 웹소켓은

초기 요청은 똑같다 일반 HTTP 요청처럼 GET 요청을 보냄 헤더에 Upgrade 관련 필드를 넣어 서버가 이걸 받아들이면 더 이상 HTTP가 아니라 새로운 프로토콜 WebSocket 즉 양방향 통신을 위해 새로운 규약을 만들어내서 사용할 수 있도록 함

그래서 결론

지예가 물어본 sse 내부에서 ping 하는 것은 무엇이나

서버가 연결이 끊기지 않도록 주기적으로 빈 이벤트를 흘려보내는 것

HTTP API 통신처럼 서로 뭔들 주고 받는 것이 아님

혹은 웹소켓처럼 ping/pong 프로토콜이 있는 건 아님

즉 서버가 연결 유지가 잘 되고 있는지 확인하는 ping 작업은 거의 헤더도 없고, 연결 유지 비용만 존재

당연 HTTP polling보다 훨씬 부하가 없음

이걸 15초마다 보내는 것이 부담이 아니라는 의미

그 15초 주기는 네트워크 환경마다 서버 상황마다 달라지게 되는데

조정을 하기도 하지만, 대부분은 그대로 두는 것 같음 default 값으로 유지

실질적으로 서버에 의미 있는 부담을 주는 것은 아님

SSE 안정성 검증 포인트

동시 연결 안정성

- 목표 동시 사용자 수: 50명
- SSE 연결이 10분 이상 안정적으로 유지되는지 검토
- TCP 연결 지속 및 ping 주기(15초) 정상 동작 확인

이벤트 순서

- DB 기반 music_statuses 이벤트가 **순서대로 전송되는지 확인**
- sent_task_ids 기반 중복 전송 방지 로직 검토
- "music_done_partial" 이벤트 순서 확인
- "music_done_all" 이벤트 마지막 전송 확인

자동 재연결

클라이언트 단절 시 SSE 자동 재연결 동작 검토

재연결 후 이벤트 누락 없이 이어받는지 확인

resume 기능과 last_event_id 기반 이벤트 이어받기 검토

Resume 기능

last_event_id 기반으로 이전 이벤트 이후부터 정상 수신되는지 확인

유효하지 않는 ID 입력 시 오류 처리 검토

재연결 후 이벤트 중복 전송 방지 로직 확인

타임아웃 처리

서버 설정 SSE_TIMEOUT 초과 시 연결 정상 종료

클라이언트 단절 감지 시 적절히 연결 종료

서버 로그 기록 확인

💡 참고

- 실제 테스트 없이도 **코드 구조와 TCP/SSE 특성을 기반으로** 안정성 판단 가능
- 50명 수준의 동시 연결은 **메모리/CPU 부담 거의 없음**
- ping 주기, polling_interval 설정으로 안정성 확보